DUOC UC — Escuela de Informática y Telecomunicaciones

Plataforma VidaSalud — Evaluación Parcial N° 3

Incorporación de Mensajería Asíncrona con RabbitMQ, Microservicios Spring Boot y Cloud (Azure / AWS)

Parte 1: Dylan Rodríguez

- Arquitectura general del proyecto (Angular, Entra ID, AWS y RabbitMQ).
- Nombres centralizados en `application.yml` y `RabbitNames`.
- Beans en `RabbitConfig` y productor `AppointmentEventPublisher`.

Parte 2: Fernando Guerrero

- Consumidores con `@RabbitListener` en Notifications.
- Confirmación manual con `basicAck` en citas y cambios de estado.
- Manejo de errores con `basicNack(requeue=false)` y cola DLQ.

Parte 3: Lucas Puentes

- Microservicio administrador REST (`ms-vidasalud-rabbit-admin`).
- Lógica encapsulada en `RabbitAdminService`.
- Validación de parámetros (HTTP 400) y pruebas realizadas.

Asignatura: Desarrollo Cloud Native I (DSY1107)

Octubre 2026

1. Arquitectura General del Sistema VidaSalud

Expositor 1: Dylan Rodríguez

```
[Frontend Angular 19 + MSAL] (Puerto 4200)
      |
      | Login y Token JWT (Microsoft Entra ID)
      v
[AWS API Gateway + ms-vidasalud-bff] (Puerto 8080)
      |
      +--> [ms-vidasalud-catalog] (Puerto 8082)
      |
      +--> [ms-vidasalud-appointments] (Puerto 8081)
                  |
                  | Publica evento después del commit (afterCommit)
                  v
            [Broker RabbitMQ] (Puertos 5672 / 15672)
                  |
                  +--> [ms-vidasalud-notifications] (Puerto 8083)
                  |    Consume con @RabbitListener y hace ACK / NACK
                  |
                  +--< [ms-vidasalud-rabbit-admin] (Puerto 8084)
                       API REST para crear/eliminar colas y exchanges
```

¿Qué agregamos en esta Evaluación 3?

- Mantuvimos funcionando todo lo anterior: el frontend Angular, la seguridad con Microsoft Entra ID y AWS API Gateway, Catalog y Appointments.
- Agregamos **RabbitMQ** para que cuando se cree una cita o cambie su estado, la notificación se procese de forma asíncrona sin bloquear la reserva.
- Creamos dos microservicios nuevos: `ms-vidasalud-notifications` (consumidor) y `ms-vidasalud-rabbit-admin` (administrador REST).

Qué decir (Dylan): "En esta evaluación extendimos nuestra plataforma VidaSalud agregando RabbitMQ para desacoplar las notificaciones del registro de citas médicas, manteniendo nuestra configuración de seguridad en Azure Entra ID y AWS."

2. Configuración Centralizada y Beans de RabbitMQ

Expositor 1: Dylan Rodríguez

Nombres centralizados en application.yml y RabbitNames

Todos los nombres de colas, exchanges y routing keys están en un solo lugar y se leen con `@ConfigurationProperties(prefix = "vidasalud.rabbit")`:

Recurso	Nombre configurado
Exchange directo	`vidasalud.appointments.exchange`
Cola de creación	`vidasalud.notifications.queue`
Routing key creación	`appointment.created`
Cola cambio de estado	`vidasalud.status.queue`
Routing key estado	`appointment.status.changed`
Dead Letter Exchange	`vidasalud.appointments.dlx`
Dead Letter Queue (DLQ)	`vidasalud.notifications.dlq`
Routing key de fallo	`appointment.failed`

Declaración de Beans en RabbitConfig.java

```
@Configuration
@EnableConfigurationProperties(RabbitNames.class)
public class RabbitConfig {
  @Bean DirectExchange appointmentsExchange(RabbitNames n) {
    return new DirectExchange(n.exchange(), true, false);
  }
  @Bean Queue notificationsQueue(RabbitNames n) {
    return QueueBuilder.durable(n.queue())
      .withArguments(Map.of(
        "x-dead-letter-exchange", n.dlx(),
        "x-dead-letter-routing-key", n.deadRoutingKey()))
      .build();
  }
  @Bean Binding createdBinding(RabbitNames n,
      DirectExchange appointmentsExchange, Queue notificationsQueue) {
    return BindingBuilder.bind(notificationsQueue)
      .to(appointmentsExchange).with(n.createdKey());
  }
}
```

Qué decir (Dylan): "Centralizamos todos los nombres en application.yml usando el record RabbitNames, y en la clase RabbitConfig declaramos los beans de Queue, Exchange y Binding para cada caso de uso."

3. Separación de Negocio y Productor en Appointments

Expositor 1: Dylan Rodríguez

Cómo separamos el negocio de la mensajería

- `AppointmentService` sigue encargándose solo de guardar la cita en H2/Oracle y validar los cambios de estado con el catálogo de boxes.
- Para no mezclar código de RabbitMQ en el servicio, creamos la clase aparte `AppointmentEventPublisher` dentro del paquete `messaging`.
- Usamos `afterCommit()` para que el evento solo se publique cuando la transacción en la base de datos ya se confirmó.

Qué decir (Dylan): "En Appointments separamos la mensajería en la clase AppointmentEventPublisher y publicamos después del commit de la base de datos para no enviar eventos si una transacción falla. Ahora continúa mi compañero Fernando."

```
// AppointmentEventPublisher.java
public void publishAfterCommit(AppointmentResponse appointment, String
routingKey) {
    Runnable send = () -> {
        Map<String, Object> event = Map.of(
            "eventId", UUID.randomUUID().toString(),
            "eventType", routingKey,
            "occurredAt", Instant.now().toString(),
            "appointment", appointment
        );
        rabbitTemplate.convertAndSend(exchange, routingKey, event, message -> {
            message.getMessageProperties().setMessageId((String)
event.get("eventId"));
            message.getMessageProperties().setContentType("application/json");

message.getMessageProperties().setDeliveryMode(MessageDeliveryMode.PERSISTENT);
            return message;
        });
    };
    if (TransactionSynchronizationManager.isActualTransactionActive()) {
        TransactionSynchronizationManager.registerSynchronization(
            new TransactionSynchronization() {
                @Override public void afterCommit() { send.run(); }
            });
    } else { send.run(); }
}
```

4. Consumidores con @RabbitListener y Confirmación Manual (ACK)

Expositor 2: Fernando Guerrero

Implementación en ms-vidasalud-notifications

- Agrupamos los consumidores en el paquete `consumer`: `AppointmentConsumer` para los eventos de citas y `DeadLetterConsumer` para la DLQ.
- Tenemos dos listeners separados con `ackMode = "MANUAL"`:
  1. `onCreated`: escucha la cola `vidasalud.notifications.queue`.
  2. `onStatus`: escucha la cola `vidasalud.status.queue`.
- Cuando `NotificationService` revisa que el JSON tiene los datos obligatorios sin errores, confirmamos el mensaje manualmente con `channel.basicAck(tag, false)`.

Qué decir (Fernando): "En el microservicio Notifications implementamos los consumidores con @RabbitListener y confirmación manual. Si el mensaje se procesa bien en NotificationService, ejecutamos basicAck para confirmar a RabbitMQ."

```
// AppointmentConsumer.java
@Component
public class AppointmentConsumer {
    @RabbitListener(queues = "${vidasalud.rabbit.queue}", ackMode =
"MANUAL")
    public void onCreated(Message message, Channel channel) throws
IOException {
        consume(message, channel);
    }

    @RabbitListener(queues = "${vidasalud.rabbit.status-queue}", ackMode =
"MANUAL")
    public void onStatus(Message message, Channel channel) throws
IOException {
        consume(message, channel);
    }

    private void consume(Message message, Channel channel) throws
IOException {
        long tag = message.getMessageProperties().getDeliveryTag();
        try {
            JsonNode event = mapper.readTree(message.getBody());
            service.process(event);
            channel.basicAck(tag, false);
        } catch (Exception ex) {
            log.error("Mensaje rechazado y dirigido a DLQ: {}",
ex.getMessage(), ex);
            channel.basicNack(tag, false, false);
        }
    }
}
```

5. Manejo de Errores con NACK y Envío a la DLQ

Expositor 2: Fernando Guerrero

¿Qué ocurre cuando un mensaje falla?

1. Si llega un mensaje incompleto o con `"simulateFailure": true`, `NotificationService` lanza una excepción.
2. El consumidor atrapa el error, lo deja registrado en los logs y ejecuta `channel.basicNack(tag, false, false)`.
3. El segundo `false` (`requeue = false`) evita que el mensaje vuelva a la misma cola en bucle.
4. Como la cola tiene configurado `x-dead-letter-exchange`, RabbitMQ mueve el mensaje a `vidasalud.appointments.dlx` y termina en la cola `vidasalud.notifications.dlq`.

Prueba del camino correcto y del envío a DLQ

```
# 1. Cita creada en POST /api/appointments -> Log en Notifications:
INFO NotificationService : Notificación generada evento=8f4b2c10...
tipo=appointment.created cita=1

# 2. Mensaje publicado con "simulateFailure": true -> Log de error:
ERROR AppointmentConsumer : Mensaje rechazado y dirigido a DLQ:
Evento rechazado para prueba de DLQ
```

En `DeadLetterConsumer` dejamos `autoStartup = "false"` para que el mensaje quede guardado en la DLQ y podamos mostrarlo en RabbitMQ Management.

> **Qué decir (Fernando):** "Cuando hay un error usamos basicNack con requeue en false para no reencolar el mensaje fallido, enviándolo directo a la DLQ y registrando el error en los logs. Ahora le doy el pase a Lucas."

6. Microservicio Administrador REST y RabbitAdminService

Expositor 3: Lucas Puentes

Endpoints expuestos en RabbitAdminController

Método	Ruta REST	Descripción
POST	/api/rabbit-admin/queues	Crea una cola nueva (201)
DELETE	/api/rabbit-admin/queues/{name}	Elimina una cola (204)
POST	/api/rabbit-admin/exchanges	Crea un exchange (201)
DELETE	/api/rabbit-admin/exchanges/{name}	Elimina un exchange (204)
POST	/api/rabbit-admin/bindings	Crea un binding (201)
DELETE	/api/rabbit-admin/bindings	Elimina un binding (204)

Lógica encapsulada en RabbitAdminService

El controlador no llama directamente a RabbitMQ, sino que delega todo a `RabbitAdminService` (que usa `AmqpAdmin` para declarar o borrar los recursos).

```
// RabbitAdminService.java
@Service
public class RabbitAdminService {
    private final AmqpAdmin admin;

    public RabbitAdminService(AmqpAdmin admin) { this.admin = admin; }

    public void createQueue(QueueRequest request) {
        admin.declareQueue(new Queue(request.name(), request.durable(),
false, false));
    }
    public void deleteQueue(String name) {
        admin.deleteQueue(name);
    }
    public void createExchange(ExchangeRequest request) {
        Exchange exchange = switch (request.type()) {
            case "direct" -> new DirectExchange(request.name(), true,
false);
            case "topic"  -> new TopicExchange(request.name(), true,
false);
            case "fanout" -> new FanoutExchange(request.name(), true,
false);
            default -> throw new IllegalArgumentException("Tipo inválido");
        };
        admin.declareExchange(exchange);
    }
}
```

Qué decir (Lucas): "Para administrar RabbitMQ por API REST creamos ms-vidasalud-rabbit-admin en el puerto 8084. Toda la lógica de creación y eliminación de colas, exchanges y bindings está encapsulada en RabbitAdminService."

7. Validación de Parámetros de Entrada y Pruebas de la API

Expositor 3: Lucas Puentes

Validación con DTOs (@NotBlank y @Pattern)

- Para evitar que se creen colas con nombres vacíos o configuraciones mal hechas, validamos los datos de entrada con `@Valid`, `@NotBlank` y expresiones regulares `@Pattern`.
- Si alguien envía un nombre en blanco `""` o un tipo de exchange inválido, `RestExceptionHandler` devuelve inmediatamente **HTTP 400 Bad Request**.
- Además, protegimos los endpoints con el filtro `AdminKeyFilter` que pide el encabezado `X-Admin-Key`.

```
public record QueueRequest(
  @NotBlank @Pattern(regexp="[a-zA-Z0-9][a-zA-Z0-9._-]{0,127}") String
name,
  boolean durable
) {}
```

Pruebas ejecutadas en PowerShell

```
$headers = @{ "X-Admin-Key" = "vidasalud-admin-local-2026-key" }

# 1. Crear cola de prueba -> Devuelve HTTP 201 y aparece en RabbitMQ
$body = @{ name = "vidasalud.test.queue"; durable = $true } |
ConvertTo-Json
Invoke-RestMethod -Uri "http://localhost:8084/api/rabbit-admin/queues"
`
  -Method POST -Headers $headers -ContentType "application/json" -Body
$body

# 2. Intentar crear cola con nombre vacío "" -> Devuelve HTTP 400
$bad = @{ name = ""; durable = $true } | ConvertTo-Json
Invoke-RestMethod -Uri "http://localhost:8084/api/rabbit-admin/queues"
`
  -Method POST -Headers $headers -ContentType "application/json" -Body
$bad
# Resultado: HTTP 400 Bad Request {"error": "Parámetros inválidos"}

# 3. Eliminar cola -> Devuelve HTTP 204 y desaparece del panel
Invoke-RestMethod -Uri "http://localhost:8084/api/rabbit-
admin/queues/vidasalud.test.queue" `
  -Method DELETE -Headers $headers
```

Qué decir (Lucas): "Validamos todos los parámetros usando anotaciones @NotBlank y @Pattern en los DTOs. Cuando intentamos crear una cola con nombre vacío, la API la rechaza con código HTTP 400."

8. Conclusión y Resumen de Pruebas

Cierre Grupal (Dylan, Fernando y Lucas)

Lo que logramos demostrar en esta entrega

- **Configuración ordenada:** Nombres centralizados en `application.yml` y beans de colas, exchanges y bindings en `RabbitConfig`.
- **Flujo real desacoplado:** Al crear o cambiar el estado de una cita en Appointments, el evento viaja por RabbitMQ y es procesado por Notifications sin afectar las reglas de negocio.
- **Control de fallos:** Uso de ACK manual cuando el evento es válido y NACK sin reencolar (`requeue=false`) para enviar los mensajes con error a la DLQ.
- **Administración dinámica:** API REST funcional con `RabbitAdminService` y validación de entrada contra datos vacíos.

Cierre de la presentación

"En resumen, nuestro principal logro fue incorporar la mensajería asíncrona con RabbitMQ a VidaSalud sin reemplazar ni romper la lógica existente de citas ni la seguridad en Azure y AWS. Probamos tanto el flujo normal de creación de citas con confirmación ACK como el desvío de errores hacia la DLQ y la administración de recursos mediante nuestra API REST validada. ¡Muchas gracias!"